

Benchmarking KV Cache Management Strategies for Long-Context LLM Inference

HPML Final Project

Ting-Feng Huang
MS in Computer Science
Columbia University
New York
th3192@columbia.edu

Hung-Kai Huang
MS in Data Science
Columbia University
New York
hh3164@columbia.edu

Yipeng Wang
MS in Data Science
Columbia University
New York
yw4623@columbia.edu

Sripad Karne
MS in Data Science
Columbia University
New York
sk5695@columbia.edu

Abstract—The Key-Value (KV) cache is the dominant memory bottleneck during autoregressive LLM inference, growing linearly with sequence length and becoming critical under concurrent multi-user workloads. We present a systematic empirical benchmark of four KV cache management strategies—Full Cache, H2O, StreamingLLM, and PagedAttention—across short-answer comprehension and long-form generation, and we further study PagedAttention-based vLLM configurations under realistic concurrent serving workloads. Our results show that eviction-based methods can achieve substantial memory reduction with no quality loss on short tasks, but diverge sharply on long-form generation, where poorly timed eviction causes catastrophic quality collapse. Under concurrent serving, KV memory emerges as the primary throughput bottleneck in the vLLM-based configurations, and engine-level design choices dominate tail latency. These findings provide concrete, workload-dependent guidance for selecting KV cache strategies in production LLM serving.

I. INTRODUCTION

A. Background

Large Language Models have demonstrated remarkable capabilities across a broad spectrum of natural language processing tasks, yet their deployment at scale introduces significant systems-level challenges during inference. A fundamental bottleneck is the memory footprint of the Key-Value (KV) cache, which grows linearly with sequence length and rapidly exhausts GPU memory capacity under concurrent multi-user workloads, constraining serving throughput and increasing hardware cost.

B. Problem Statement

Existing KV cache management techniques are typically evaluated under inconsistent settings on isolated single-stream benchmarks, with little visibility into concurrent serving behavior. A direct, controlled comparison across methods and workload regimes is missing.

C. Scope and Contributions

We benchmark four representative KV cache management strategies under a unified, fairness-controlled pipeline across short- and long-context tasks, and separately evaluate vLLM-based concurrent serving behavior under a realistic mixed

workload. Contributions: (i) a unified benchmarking harness; (ii) a controlled single-request memory–quality comparison; (iii) a concurrent serving analysis across multiple engine configurations and arrival rates.

II. LITERATURE REVIEW

Transformer-based language models build on the self-attention architecture introduced by Vaswani et al. [1]. At serving time, autoregressive decoding introduces a growing KV cache, shifting long-context inference toward memory-capacity and memory-bandwidth bottlenecks. PagedAttention [2] draws an analogy to OS virtual memory management, partitioning the KV cache into fixed-size blocks to eliminate fragmentation and support high-throughput concurrent scheduling. StreamingLLM [3] retains a small set of attention sink tokens alongside a fixed recent window, enabling bounded-memory generation over arbitrary sequence lengths. H2O [4] takes a data-driven approach, pruning low-importance KV entries by cumulative attention score within a configurable budget. Complementary techniques such as quantization and fused attention kernels operate orthogonally to these policies; this study isolates cache management specifically to enable controlled comparison.

III. METHODOLOGY

This section describes the four KV cache management strategies under study (Section III-A), the model and datasets (Sections III-B and III-C), the two experiments designed to expose their trade-offs (Section III-D), the supporting infrastructure (Section III-E), and the evaluation metrics (Section III-F). The scope is purely *inference-time*: model weights are not modified, so any quality difference is attributable to cache management alone.

A. KV Cache Management Strategies

We evaluate four strategies that span the design space of KV cache management. All four are wrapped in a common `BaseMethod` interface (`setup`, `generate`, `teardown`) and emit a uniform per-request record carrying token counts,

time-to-first-token (TTFT), total latency, peak KV memory, and method-specific metadata. KV memory is reported by summing `nelement × element_size` over every key and value tensor in the cache, so that methods with very different cache representations are compared on the same byte-level basis. Table I summarizes their key properties.

TABLE I
KV CACHE MANAGEMENT STRATEGIES COMPARED.

Method	Backend	Cache structure and retention policy
Full Cache	HF	DynamicCache; retain all tokens.
H2O	HF eager	DynamicCache; retain top- K attention-heavy tokens plus last R tokens.
StreamingLLM	HF	DynamicCache; retain first S sink tokens plus last R tokens.
PagedAttention	vLLM 0.8.5	16-token block pool; free cache at block granularity.

Full Cache is the HuggingFace baseline: every key/value pair produced during generation is retained in the standard DynamicCache and consumed by the fused SDPA kernel. It establishes the upper bound on both quality and KV memory and serves as the reference against which the three reduction strategies are measured.

H2O [4] accumulates per-layer cumulative attention scores at each decode step and, once the cache exceeds a budget of $K + R$ tokens, evicts the lowest-scoring non-recent tokens. The fused SDPA kernel does not expose attention weights, so decode falls back to the eager attention implementation with `output_attentions=True`, while prefill remains on SDPA. We use $K = R = 64$, giving a 128-token budget.

StreamingLLM [3] retains the first S “attention sink” tokens together with the most recent R tokens and drops everything in between. We implement this directly on DynamicCache: in transformers ≥ 4.36 , RoPE is baked into keys before they enter the cache, so no attention-forward patch is required—only the `cache_position` of the new query must be overridden so that it receives the correct rotation. We fix $S = 4$ and sweep $R \in \{64, 1024, 4096\}$ to expose the memory–quality knob.

PagedAttention [2], as implemented in vLLM 0.8.5, allocates the cache as fixed-size physical blocks (16 tokens each) and indirects through a per-request page table, decoupling logical contiguity from physical contiguity. To keep the comparison against the HuggingFace methods fair, we disable optimizations that have no analog in HF (`enforce_eager=True`, `enable_prefix_caching=False`); their effect is studied separately in Experiment 2 by sweeping the v0/v1 engines against graph and eager modes (Table IV(b)).

A fifth variant grafting the StreamingLLM sink-recent eviction policy onto vLLM’s block manager was implemented and verified in single-stream mode but exhibits a CUDA illegal-memory-access race between the block-free path and in-flight kernels under concurrent serving. We document this failure

mode but exclude the variant from the concurrent-serving results.

B. Model

All experiments use Qwen2.5-7B [5] in float16: a decoder-only Transformer with $L = 28$ layers, 28 query heads, and $H_{kv} = 4$ key/value heads under grouped-query attention (GQA) [6]. Float16 weights occupy approximately 14 GB of GPU memory, leaving the remainder of a 48 GB GPU available for the KV cache. The aggressive $7\times$ reduction in KV heads relative to query heads makes Qwen2.5-7B a realistic stress test for cache management strategies, since the per-token KV footprint

$$\text{KV}(T) = 2 \cdot L \cdot T \cdot H_{kv} \cdot d_h \cdot b \quad (1)$$

still grows linearly in the cache occupancy T and dominates memory beyond a few thousand tokens. With $d_h = 128$ and $b = 2$ bytes (float16), each cached token contributes 57,344 bytes, or approximately 56 KiB, of KV state. Greedy decoding (temperature=0) is used throughout to make outputs deterministic and to focus the comparison on cache management rather than sampling stochasticity.

C. Datasets

Three datasets, summarized in Table II, span short-context multiple choice (MMLU), long-context multiple choice (LongBench v2), and long-context generation (GovReport). A single ingestion pipeline downloads each dataset, formats prompts as shown, tokenizes with the Qwen2.5-7B tokenizer, filters samples to a per-experiment token-length range, caps per-subject or per-domain counts, and emits a unified record (prompt, reference, source, subset, token count). Identical formatted prompts across all four methods are the necessary condition for a fairness-controlled comparison.

TABLE II
DATASETS AND PROMPT FORMATS.

Dataset	Source	Prompt format	Reference
MMLU	cais/mmlu	5-shot, lettered choices, Answer: suffix	A/B/C/D
LongBench v2	THUDM/LongBench-v2	context + question + A/B/C/D	A/B/C/D
GovReport	ccdv/govreport-summarization	“Please read ... Summary:”	ref. summary

D. Experimental Design

We design two experiments to disentangle the controlled memory–quality trade-off (Experiment 1) from realistic concurrent serving behavior (Experiment 2).

1) *Experiment 1 – single-request grid*: Greedy decoding at batch size one, after CUDA warm-up, with timing via `perf_counter` and an explicit `cuda.synchronize()` before each timestamp. Four sub-benchmarks (Table III) form a 2×2 grid over input length (short MMLU vs. long LongBench/GovReport) and generation length (10 vs. 512 new

tokens). The grid distinguishes which methods break on long input, long output, or both.

2) *Experiment 2 – concurrent serving*: A 500-request trace replays a heterogeneous bucket mix (Table IV(a)) under Poisson arrivals at four offered loads. Inter-arrival times are sampled from the exponential distribution

$$t_{i+1} = t_i - \frac{\ln U_i}{\lambda}, \quad U_i \sim \text{Uniform}(0, 1), \quad (2)$$

with $\lambda \in \{0.5, 1.0, 2.0, 4.0\}$ req/s. The same trace file is reused across all methods and rates: arrival timestamps are scaled linearly to vary λ while preserving the inter-arrival distribution. After bucket assignment a deterministic shuffle (seed 42) interleaves long and short requests so that head-of-line blocking and KV pressure are realistic. The bucket mix (50/30/15/5) stresses every regime simultaneously: a short-tail majority sensitive to TTFT, a medium body that consumes most decode tokens, and a long / very-long tail that dominates KV memory and GPU time.

Requests are submitted to `vllm.AsyncLLMEngine` as independent `asyncio` tasks that sleep until their arrival timestamp before calling `engine.generate`, leaving cross-request decode batching to vLLM’s continuous-batching scheduler. For PagedAttention we additionally sweep four engine configurations (Table IV(b)) crossing v0/v1 with graph/eager mode; all share `dtype=float16`, `max_model_len=34000`, `gpu_memory_utilization=0.9`, `block_size=16`, and disabled prefix caching. A fresh engine is constructed per arrival rate so that KV-pool state from a previous rate cannot contaminate the next.

E. Experimental Infrastructure

All experiments run on Columbia’s Insomnia HPC cluster. Each Slurm job requests a single 48 GB GPU on a node running CUDA 12.3, Python 3.11, PyTorch 2.6, transformers 4.57, and vLLM 0.8.5. Experiment 1 (controlled single-request benchmarks) runs on NVIDIA RTX L40S, while Experiment 2 (concurrent serving workload) runs on NVIDIA RTX A6000; both GPUs expose 48 GB of HBM, which keeps KV-memory measurements directly comparable across the two experimental regimes. Code, virtual environments, and model weights are staged on scratch storage; weights are downloaded once and reused across jobs in offline mode.

Reproducibility is enforced at multiple layers: greedy decoding (temperature=0); a fixed RNG seed (42) for trace generation; identical formatted prompts across methods, guaranteed by the shared pipeline and tokenizer; exact package version pins; and per-run JSON results committed under `results/`. Figures are regenerated from these JSON artifacts without rerunning GPU jobs, which decouples result analysis from compute availability.

F. Evaluation Metrics

Quality. Accuracy on MMLU and LongBench is reported after extracting the predicted answer via a structured-pattern cascade (`Answer: prefix → trailing letter → leading letter`

→ last standalone letter). On GovReport we report ROUGE-L F1, computed from LCS-based recall and precision over whitespace-tokenized lower-cased text.

Latency. Experiment 1 reports TTFT, decode latency (total minus TTFT, isolating the autoregressive phase where eviction operates), end-to-end latency, and decode throughput in tok/s. Experiment 2 reports p50/p95/p99 of TTFT and end-to-end latency, both overall and *per bucket*, together with request-rate and token-rate throughput over the full 500-request trace. The per-bucket breakdown is essential because aggregate percentiles can hide catastrophic regressions on the long-input tail.

KV memory. For the HuggingFace methods we report the per-request peak from the unified accounting routine. For vLLM, which pre-allocates the entire block pool at startup, the runtime delta is uninformative; we instead report the analytical lower bound from (1) at the realized cache occupancy and, for concurrent runs, observed block-pool utilization sampled every 0.5 s by polling the v0 scheduler’s block manager. We separate the prefill peak from the overall peak so that prefill cost is comparable across methods that diverge only during decode. vLLM v1 runs the engine in a subprocess that exposes no block-manager handle, so we fall back to `nvidia-smi` GPU-memory polling and mark the corresponding rows N/A.

Reliability. The number of CUDA out-of-memory events (n_{oom}) is tracked per bucket and overall. The Experiment 1 runner catches `torch.cuda.OutOfMemoryError`, calls `empty_cache()`, records a zero-metric placeholder, and continues. This lets us distinguish methods that survive long-context inputs from those that silently degrade.

IV. EXPERIMENTAL RESULTS

The setup of each experiment is specified in Section III-D; we report results in two parts, corresponding to the controlled single-request grid (Section IV-A) and the concurrent-serving workload (Section IV-B).

A. Single-Request Memory–Quality Trade-off

We first evaluate all methods under a single-request setting to isolate the effect of KV cache management without system-level interference.

Short-answer tasks. On MMLU and LongBench short-answer tasks, all methods achieve identical accuracy (56.7% and 40.9%, respectively). However, this does not mean eviction is absent. KV cache trimming is actively applied, and significant memory reduction is observed. In particular, StreamingLLM reduces decode-phase KV memory from 732 MB to 3.7 MB, achieving up to 198× reduction compared to the full cache baseline, while H2O achieves moderate reduction.

Despite identical output quality, system-level differences already emerge. H2O shows approximately 25% lower throughput due to its reliance on unfused attention kernels for computing attention weights during eviction.

These results indicate that short-answer tasks mainly serve as a sanity check: they confirm that eviction does not affect

TABLE III
EXPERIMENT 1: SINGLE-REQUEST SUB-BENCHMARKS.

Sub-benchmark	Config	Dataset	n	Input tokens	Max new tokens	Quality metric
MMLU short	experiment1_short.yaml	MMLU 5-shot	150	128–512	10	Accuracy
LongBench short	experiment2_long.yaml	LongBench v2	22	8k–16k	10	Accuracy
LongBench explain	experiment2_long_explain.yaml	LongBench v2	22	8k–16k	512	Accuracy
GovReport summ.	experiment2_summarization.yaml	GovReport	50	4k–16k	512	ROUGE-L

TABLE IV
EXPERIMENT 2 CONCURRENT-SERVING SETUP.

(a) 500-request workload mix				
Bucket	%	Source	Tokens	max_new
short	50	MMLU	0–512	8
medium	30	GovReport	1k–4k	256
long	15	GovReport	4k–16k	512
very_long	5	LongBench v2	16k–32k	10

(b) vLLM engine configurations for PagedAttention			
Config	Engine	CUDA graphs	Chunked prefill
v0_graph	v0	on	off
v0_eager	v0	off	off
v1_graph	v1	on	on, 2k tok
v1_eager	v1	off	on, 2k tok

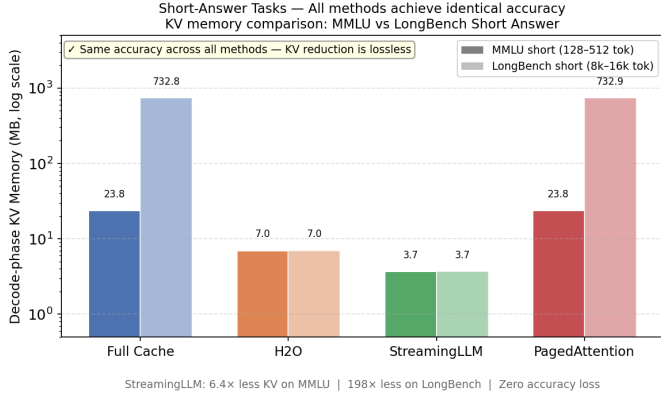


Fig. 1. KV memory usage during decoding across methods. StreamingLLM achieves up to 198× reduction compared to Full Cache while maintaining identical accuracy on short tasks.

correctness, while revealing differences in memory efficiency and runtime overhead.

Long-form generation. We next evaluate long-form generation using the GovReport dataset (4k–16k input tokens, 512 output tokens), where eviction directly affects all decoding steps.

In this setting, clear differences appear. H2O collapses completely, achieving ROUGE-L = 0.000 across all tested budgets. This suggests that attention-score-based eviction is brittle in this setting: early decode steps provide limited output-side attention history, so eviction can remove context

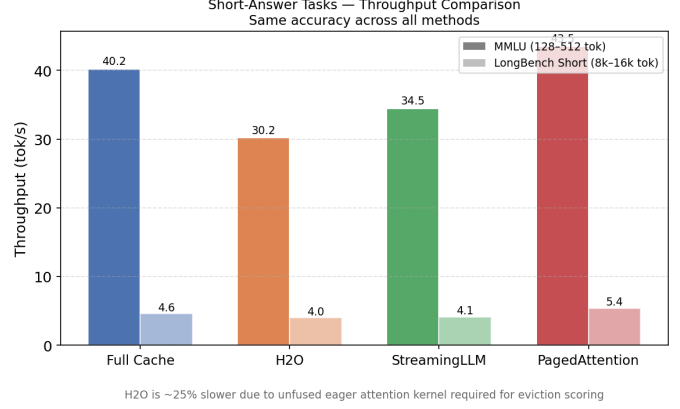


Fig. 2. Throughput comparison on short-answer tasks. H2O shows lower throughput due to unfused attention kernel overhead, while other methods achieve similar performance.

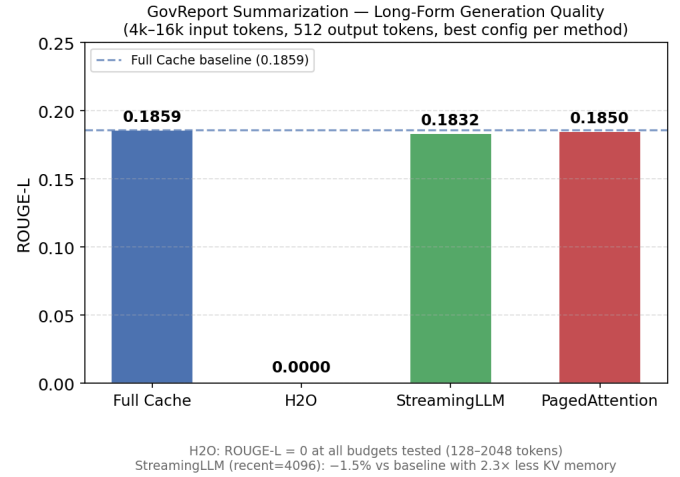


Fig. 3. ROUGE-L comparison on GovReport summarization. H2O collapses to zero, consistent with brittle early eviction decisions, while StreamingLLM achieves near-baseline performance with sufficient window size.

that later tokens require.

In contrast, StreamingLLM degrades more gracefully. With a sufficiently large window (recent = 4096), it achieves ROUGE-L = 0.1832 compared to the full cache baseline of 0.1859 (only 1.5% degradation), while reducing KV memory by approximately 2.3×.

PagedAttention, which does not perform eviction, matches the full cache baseline, confirming that the evaluation pipeline

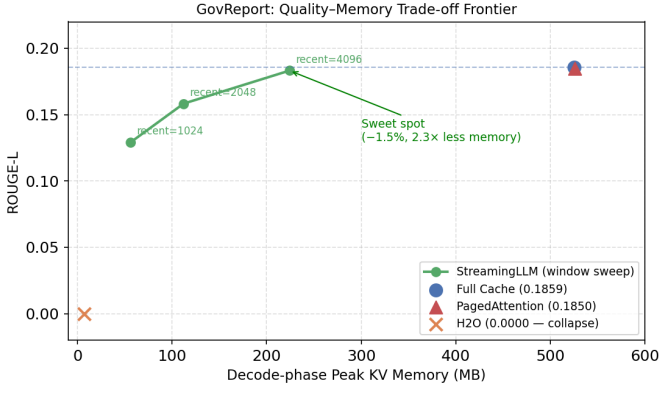


Fig. 4. Quality-memory trade-off for StreamingLLM with different window sizes. Larger windows improve quality while smaller windows reduce memory usage.

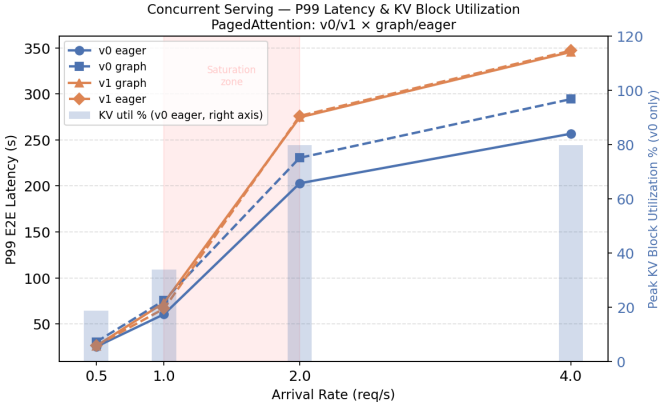


Fig. 5. P99 latency and KV utilization versus arrival rate. KV memory becomes the primary bottleneck as load increases.

is correct.

Quality-memory trade-off. These results reveal a clear trade-off between memory usage and output quality.

For StreamingLLM, the window size directly controls this trade-off. A smaller window leads to larger memory savings but worse quality, while a larger window preserves quality at the cost of higher memory usage. For example, $\text{recent} = 1024$ results in about 30% ROUGE-L drop but $9.3\times$ memory reduction, while $\text{recent} = 4096$ achieves near-baseline quality with $2.3\times$ reduction.

Overall, this experiment shows that eviction effectiveness depends on how well the retained context matches the task requirements.

B. Concurrent Serving Behavior

We further evaluate vLLM-based serving configurations under a realistic concurrent workload with 500 requests and Poisson arrivals.

Latency and KV memory pressure.

As the request rate increases, the system saturates between 1.0 and 2.0 requests per second. KV cache utilization grows

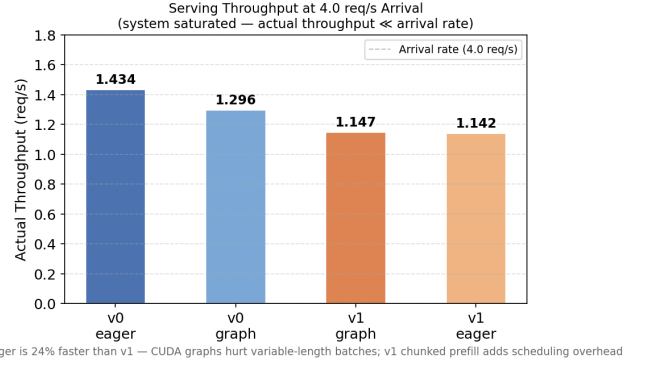


Fig. 6. Throughput comparison under concurrent serving. v0 eager achieves the best performance among all configurations.

from approximately 18.7% at low load (0.5 req/s) to 79.7% at saturation, indicating that KV memory becomes the primary bottleneck.

Among all configurations, v0 eager achieves the best performance, with the highest throughput (1.43 req/s) and lowest tail latency. Output quality remains identical across configurations, meaning the observed differences are purely system-level effects.

System-level effects.

We observe that system design choices significantly affect performance. CUDA graphs do not provide benefits in this workload because variable-length inputs prevent effective graph reuse, introducing overhead instead of speedup.

In addition, vLLM v1 performs worse than v0 in our setup, likely because chunked prefill splits long inputs into multiple scheduling steps and adds overhead for this workload.

We also attempted to integrate StreamingLLM into concurrent serving, but encountered instability due to memory management issues, highlighting practical challenges when combining eviction strategies with dynamic allocation.

V. DISCUSSION

A. Interpretation

H2O’s collapse on GovReport (ROUGE-L = 0.000) appears to result from committing to eviction before reliable output-side attention patterns are available. StreamingLLM sidesteps this by anchoring on sink and recent tokens, which matches the structure local-attention decoders actually use: $R = 4096$ stays within 1.5% of full cache at $2.3\times$ less memory, while $R = 1024$ trades 30% ROUGE-L for $9.3\times$ savings. The window R therefore has to scale with output length, not just input length, since eviction error compounds across decode steps.

Kernel choice often outweighs the policy itself. H2O needs raw attention scores, which forces the eager path because fused SDPA does not expose them; this costs $O(T^2)$ prefill memory and roughly 25% throughput on short-answer tasks before eviction even begins. Any score-based eviction inherits this constraint.

In concurrent serving the dominant variable is engine configuration. v0 eager reaches 1.434 req/s at saturation, 10%

above v0 graph and 24% above v1 graph, with identical outputs across configurations. CUDA graphs need shape stability that the 50/30/15/5 bucket mix violates; v1’s chunked prefill may amortize long inputs across scheduling steps, but in this single-GPU heterogeneous workload it adds per-step overhead. PagedAttention’s win comes from the block pool and continuous batching, not from any uniformly preferable kernel flag.

B. Limitations

All experiments use a single 48 GB GPU. H2O’s eager prefill at $O(T^2)$ prevents budgets large enough to test recovery on 4k–16k inputs, so the GovReport collapse cannot be cleanly separated from the budget ceiling. LongBench explain-format prompts are also a weak signal: Full Cache itself only reaches 27.3%, leaving little headroom to resolve eviction-induced quality loss.

We evaluate one model family (Qwen2.5-7B, float16) and do not sweep KV quantization. StreamingLLM variant is verified single-stream but excluded from concurrent results due to a reproducible CUDA illegal-memory-access race between block-free and in-flight kernels. All serving results are single-GPU, which is not v1’s intended regime.

C. Future Work

Combining eviction with paged allocation is the most direct next step: a synchronization protocol between the block manager and the eviction hook would unlock StreamingLLM’s memory savings at PagedAttention’s throughput. KV quantization (FP8, INT4) is orthogonal and compounds with eviction since it reduces per-token bytes rather than retained tokens. Repeating the H2O sweep on an 80 GB H100 or H200 would resolve whether the GovReport collapse is a policy failure or a budget artifact. Finally, multi-GPU tensor-parallel serving would re-evaluate v1 in its intended regime and lift the 34k context ceiling.

VI. CONCLUSION

We built a unified harness for KV cache management and evaluated four strategies (Full Cache, H2O, StreamingLLM, PagedAttention) on Qwen2.5-7B under a single-request grid. We additionally evaluated PagedAttention/vLLM configurations on a 500-request concurrent trace.

Eviction does not affect short-answer accuracy but does affect memory and runtime: StreamingLLM reaches $198\times$ KV reduction; H2O pays 25% throughput for eager attention. On long-form generation H2O collapses to ROUGE-L = 0.000, StreamingLLM stays within 1.5% of baseline at $R = 4096$, and PagedAttention matches Full Cache. Under concurrency, engine configuration dominates: v0 eager beats every alternative by 10–24%, with KV utilization rising from 18.7% to 79.7% as load increases.

The practical rule is workload-dependent. Use StreamingLLM with $R \geq 4096$ when memory binds. Use vLLM v0 eager for heterogeneous online serving on a single GPU. Avoid score-based eviction on long contexts

unless eager-prefill memory is not a constraint. More broadly, KV strategy and serving engine cannot be evaluated in isolation: a policy optimal in single-stream becomes infeasible under concurrency, and a flag that helps uniform batches hurts mixed ones.

REFERENCES

- [1] A. Vaswani *et al.*, “Attention Is All You Need,” in *NeurIPS*, 2017.
- [2] W. Kwon *et al.*, “Efficient Memory Management for Large Language Model Serving with PagedAttention,” in *SOSP*, 2023.
- [3] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, “Efficient Streaming Language Models with Attention Sinks,” *arXiv preprint arXiv:2309.17453*, 2023.
- [4] Z. Zhang *et al.*, “H2O: Heavy-Hitter Oracle for Efficient Generative Inference of Large Language Models,” in *NeurIPS*, 2023.
- [5] Qwen Team, “Qwen2.5 Technical Report,” 2024.
- [6] J. Ainslie *et al.*, “GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints,” *arXiv preprint arXiv:2305.13245*, 2023.